COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	08/20/26	**DATE SUBMITTED:**	08/20/26
NAME:	Hare, Andrei Angelo D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ASSIGNMENT NO. 6.1 Linear and Binary Search

1. PROCEDURE OUTPUT

What procedures did you perform?

Answer the following questions:

1. What is a search tree in data structures?

   A search tree is a type of data structure where the nodes are connected in a tree-like form. It is used to store data in an organized way so searching, inserting, and deleting can be done faster. Each node contains a value, and the values are arranged in a specific order. For example, in a binary search tree, smaller values are placed on the left side while bigger values are placed on the right side. Because of this arrangement, we don't need to check every node one by one when searching for a value.

   Source / Reference: GeeksforGeeks - Binary Search Tree in C++

2. What are the Different types of search algorithm in data structures? Differentiate each type of search.

   Search algorithms can be divided into two main types:

   a. **Sequential search**
      Checks the data one by one from the beginning until the target is found or there are no more items to check. It does not require the data to be sorted.
      Examples: Linear Search and Sentinel Linear Search
   b. **Interval or Division-Based Search**
      Works by reducing the search area into smaller parts to find the target faster, and it usually needs sorted data.
      Examples: Binary Search, Jump Search, Interpolation Search, and Exponential Search

Feature	Sequential Search (e.g., Linear Search)	Interval Search (e.g., Binary Search)
Data Requirement	Works on both sorted and unsorted data.	Requires sorted data.
Search Mechanism	Inspects elements sequentially one by one.	Repeatedly divides search space based on comparisons.
Data Structure Suitability	Efficient for Linked Lists and small arrays.	Best for contiguous access structures (Arrays).

Source / Reference: GeeksforGeeks - Searching Algortihms
Linear Search vs Binary Search

3. What operations / implementations can be performed using binary and linear search operations?

Search algorithms can perform different operations depending on what we need to find.

- **Search or key lookup** is used to check if a certain value exists and find its position.
- **Boundary and range queries** can find values like the lower bound, upper bound, floor, or ceiling.
- **Count or frequency finding** is used to count how many times a value appears, especially in sorted data.
- **Minimum and maximum lookup** is used to find the smallest or largest value in the data.

For implementations:

- **Linear Search Implementations** can be used for unsorted lists, small arrays, or simple lookup tables.
- **Binary Search Implementations** is useful for sorted data because it can reduce the search area and find the value faster. It is also used in things like C++ search functions, autocomplete systems, database indexes, and other applications.

Source / Reference: GeeksforGeeks - Searching Interview Questions

4. What are the advantages in using binary search tree as data structure?

The binary search tree has several advantages as a data structure. It can make searching, inserting, and deleting faster when the tree is balanced. Another advantage is that the values are already arranged in order, so we can get sorted data by using in-order traversal. It also grows or becomes smaller dynamically because nodes are added or removed when needed. It is also useful for finding the minimum, maximum, nearest values, and values within a certain range.

Source / Reference:GeeksforGeeks - Advantages and Disadvantages of Tree

5. Give an example program using binary search and Linear search.

C/C++

```
#include <iostream>
#include <vector>
using namespace std;

// Function to perform Linear Search
int linearSearch(const vector<int>& arr, int target) {
    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] == target) {
            return i; // Target found at index i
        }
```

```
    }
    return -1; // Target not found
}

// Function to perform Binary Search (Array must be sorted)
int binarySearch(const vector<int>& arr, int target) {
    int left = 0;
    int right = arr.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid; // Target found at index mid
        }
        if (arr[mid] < target) {
            left = mid + 1; // Search right half
        } else {
            right = mid - 1; // Search left half
        }
    }
    return -1; // Target not found
}

int main() {
    vector<int> sortedData = {10, 20, 30, 40, 50, 60, 70};
    int target = 40;

    int linearResult = linearSearch(sortedData, target);
    if (linearResult != -1)
        cout << "Linear Search: Found " << target << " at index " << linearResult
<< endl;
    else
        cout << "Linear Search: Element not found" << endl;

    int binaryResult = binarySearch(sortedData, target);
    if (binaryResult != -1)
        cout << "Binary Search: Found " << target << " at index " << binaryResult
<< endl;
    else
        cout << "Binary Search: Element not found" << endl;

    return 0;
}
```

The program compares Linear Search and Binary Search using the sorted list {10, 20, 30, 40, 50, 60, 70}. The target value is 40.

If we use Linear Search, the program checks each number from the beginning until it finds 40. Since 40 is at index 3, it returns index 3. While in Binary Search, It first checks the middle value, which is 40. Since it is already the target, the search stops and returns index 3. The main() function calls both search functions and displays their results. If the target is not found, both functions return -1.

The main difference is that Linear Search checks the values one by one, while Binary Search keeps dividing the search range into smaller parts. Binary Search is more efficient for sorted data because it can reduce the search area faster.

Source / Reference: Introduction to Searching, Binary Search

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Quillbot, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Summarization, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It only helped me to improve my writing and did not replace my original thinking as I used the sources as a reference for my answers and explained them based on my own understanding.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.